

LAB 2 · 시프트 레지스터

작성일 2026. 09. 13. · 이해리

빈 템플릿 → 직접 작성 → Icarus 시뮬레이션 → Vivado 구현

목차

프로젝트 시작

RTL·TB·XDC 작성

VS Code 실행과 파형

실험 전 레포트

Vivado 구현

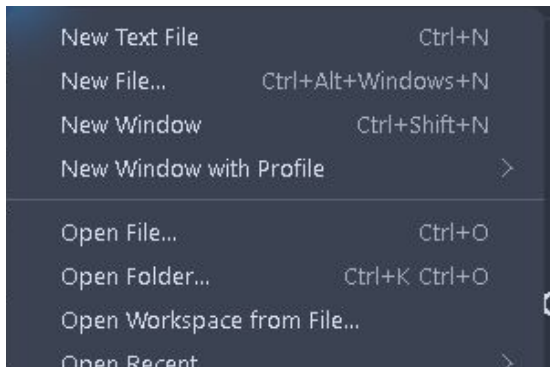
실험 후 레포트

v2.0.1 템플릿을 새 이름으로 clone

Windows PowerShell에서 한 줄씩 실행한다.

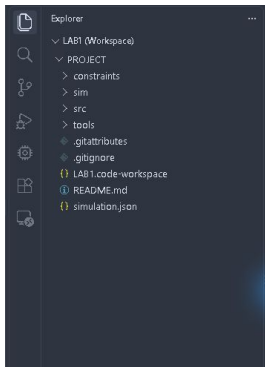
```
git clone --branch v2.0.1 `
https://github.com/Glaysia/fpga-lab-template.git `
lab2_04_shift_register
cd lab2_04_shift_register
git switch -c main
code LAB1.code-workspace
```

새 창과 워크스페이스



Ctrl+Shift+N으로 새 창을 연다. File → Open Workspace from File... → 자신의 실습 폴더 → LAB1.code-workspace. 화면은 v2.0.0 빈 템플릿의 공통 예시다. v2.0.1에서도 여는 방법은 같고 .slang 설정이 추가되어 있다.

빈 템플릿 확인



clone 직후에는 정답 코드가 없다. Explorer의 `src·sim·constraints`를 펼쳐 빈 파일을 확인하고 뒤의 내용을 작성한다. 화면은 v2.0.0 빈 템플릿의 공통 예시다. v2.0.1에서도 여는 방법은 같고 `.slang` 설정이 추가되어 있다.

workspace와 추천 확장

1. lab2_04_shift_register 폴더에서 LAB1.code-workspace 선택 → Open.
2. Explorer에 PROJECT 하나와 src.sim.constraints.tools가 보이는지 확인.
3. Extensions → @recommended를 검색.
4. slang.VaporView.vscode-pdf의 Install을 누른다.
5. Python과 Icarus 실행 파일은 별도로 설치되어 있어야 한다.

v2.0.1 편집기와 시뮬레이터

1. slang 확장·서버는 0.3.0을 기준으로 확인한다. 추천 목록은 버전을 고정하지 않는다.
2. 템플릿의 .slang/server.json은 src.sim 탐색용이다. 새 RTL은 simulation.json에도 직접 등록한다.
3. Ctrl+Shift+P → slang: Show Output. 서버 오류가 있으면 README의 버전 확인·재시작 절차를 따른다.
4. slang.path에 다른 사람의 설치 경로를 복사하지 않는다. 직접 지정할 때는 User 설정에 자신의 경로를 쓴다.
5. VS Code 02 Simulate는 Icarus, Vivado Run Behavioral Simulation은 XSim이다.

템플릿 도구 설치·오류 해결 안내

시뮬레이션 도구 확인

1. Terminal → Run Task... → 01 Check tools.
2. Git·Python·iverilog·vvp의 버전이 출력되는지 확인한다.
3. 도구를 찾지 못하면 설치 경로를 PATH에 추가하고 VS Code를 다시 연다.

```
git --version  
python --version  
iverilog -V  
vvp -V
```


빈 파일에서 시작

1. src/design.v를 우클릭 → Rename. 첫 RTL 파일명으로 바꾼다.
2. 추가 RTL은 src를 우클릭 → New File로 만든다.
3. sim/tb_design.v를 이번 TB 파일명으로 바꾼다.
4. constraints/pins.xdc를 이번 XDC 파일명으로 바꾼다.
5. 뒤의 코드 화면을 보며 작성하고 File → Save All.

이번 프로젝트에 작성할 파일

역할	경로
src	src/shift_register4.v
src	src/input_frontend.v
src	src/lab2_shift_register.v
sim	sim/tb_shift_register4.sv
constraints	constraints/lab2_shift_register.xdc

simulation.json의 세 항목

`sources`: 앞 표의 `src/` 파일 경로를 배열로 넣는다.

`testbench`: `sim/tb_shift_register4.sv`

`simulation_top`: `tb_shift_register4`

큰따옴표·대괄호·심표를 확인한다. `module` 이름에는 파일 확장자를 붙이지 않는다.

simulation.json · 1-9행

simulation.json를 열고 해당 구간을 직접 입력한다.

```
{ } LAB1.code-workspace M × shift_register4.v U × lab2_shift_register.v U ×  
{ } simulation.json  
1  {  
2    "sources": [  
3      "src/shift_register4.v",  
4      "src/input_frontend.v",  
5      "src/lab2_shift_register.v"  
6    ],  
7    "testbench": "sim/tb_shift_register4.sv",  
8    "simulation_top": "tb_shift_register4"  
9  }
```

한 비트씩 받아 네 비트를 기억한다

value[3:0]은 네 비트를 저장한다. enable=1인 clk 상승 에지마다 serial_in을 value[3]에 넣는다.

이전 value[3], value[2], value[1]은 각각 value[2], value[1], value[0]으로 이동한다. 이전 value[0]은 버린다.

rst=1이면 0으로 초기화한다. rst=0, enable=0이면 입력이 변해도 저장값을 유지한다.

다음 상태를 먼저 계산

rst	enable	다음 value[3:0]
1	무관	0000
0	0	이전 value 유지
0	1	{serial_in, 이전 value[3:1]}

예: value=0100, serial_in=1이면 다음 value=1010이다. 시프트 방향을 화면의 좌우가 아닌 비트 번호로 설명한다.

중괄호와 nonblocking 대입

`{serial_in, value[3:1]}`은 1비트와 3비트를 이어 붙인 4비트다.
덧셈이나 네 번의 순차 실행이 아니다.

`value <= ...`의 오른쪽은 상승 에지 직전 저장값에서 계산한다. 네 비트는 같은 에지의 nonblocking 갱신으로 바뀐다.

한 에지에 입력 비트가 모든 단계를 통과하지 않는다. 네 번 enable이 걸려야 입력 이력이 네 단계에 채워진다.

shift_register4.v · 1-11행

src/shift_register4.v를 열고 해당 구간을 직접 입력한다.

```
{ } LAB7.code-workspace M × shift_register4.v U ×
src > shift_register4.v > ...
1  timescale 1ns/1ps
2  // Serial input enters bit 3 and moves toward bit 0.
3  module shift_register4 (
4      input wire clk, rst, enable, serial_in,
5      output reg [3:0] value
6  );
7      always @(posedge clk) begin
8          if (rst) value <= 4'd0;
9          else if (enable) value <= {serial_in, value[3:1]};
10     end
11 endmodule
```


input_frontend.v · 1-16행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
1  timescale 1ns/1ps
2  // Main clock is 1 kHz: STABLE_CYCLES=20 means 20 ms.
3  // Set switches first, then press the button; the bus is not an atomic CDC.
4  module input_frontend #(parameter integer STABLE_CYCLES = 20) (
5      input wire clk, rst, button,
6      input wire [7:0] sw,
7      output wire reset,
8      output reg press,
9      output wire [7:0] switches
10 );
11     (* ASYNC_REG = "TRUE" *) reg [1:0] reset_pipe;
12     (* ASYNC_REG = "TRUE" *) reg button_meta, button_sync;
13     (* ASYNC_REG = "TRUE" *) reg [7:0] sw_meta, sw_sync;
14     localparam integer WIDTH = (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
15     reg [WIDTH-1:0] count;
16     reg accepted;
```

input_frontend.v · 17-32행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
17 assign reset = reset_pipe[1];
18 assign switches = sw_sync;
19 // Asynchronous assertion; release follows two main-clock edges.
20 always @(posedge clk or posedge rst) begin
21     if (rst) reset_pipe <= 2'b11;
22     else reset_pipe <= {reset_pipe[0], 1'b0};
23 end
24 always @(posedge clk or posedge reset) begin
25     if (reset) begin
26         button_meta <= 0; button_sync <= 0;
27         sw_meta <= 0; sw_sync <= 0;
28     end else begin
29         button_meta <= button; button_sync <= button_meta;
30         sw_meta <= sw; sw_sync <= sw_meta;
31     end
32 end
```

input_frontend.v · 33–46행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
33     always @(posedge clk or posedge reset) begin
34         if (reset) begin
35             count <= 0; accepted <= 0; press <= 0;
36         end else begin
37             press <= 0;
38             if (button_sync == accepted) count <= 0;
39             else if (count == STABLE_CYCLES - 1) begin
40                 accepted <= button_sync;
41                 count <= 0;
42                 press <= button_sync;
43             end else count <= count + 1'b1;
44         end
45     end
46 endmodule
```

lab2_shift_register.v · 1-16행

src/lab2_shift_register.v를 열고 해당 구간을 직접 입력한다.

```
{ } LAB1.code-workspace M × shift_register4.v U × lab2_shift_register.v U ×
src > lab2_shift_register.v
1 | timescale 1ns/1ps
2 // Combo II-DLD S75. Main clock B6 MUST be set to 1 kHz.
3 // K4: reset. M8: step button. DIPSW1..8 = sw[7]..sw[0].
4 module lab2_shift_register #(parameter integer STABLE_CYCLES = 20) (
5     input wire clk, rst, button,
6     input wire [7:0] sw,
7     output wire [7:0] led
8 );
9     wire reset, press;
10    wire [7:0] switches;
11    input_frontend #(STABLE_CYCLES(STABLE_CYCLES)) inputs(
12        clk, rst, button, sw, reset, press, switches);
13    wire [3:0] value;
14    shift_register4 core(clk, reset, press, switches[7], value);
15    assign led = {4'b0000, value};
16 endmodule
```

버튼 한 번이 한 단계가 되도록

input_frontend는 비동기 입력을 주 클록에 동기화하고 버튼이 20회 연속 안정된 뒤 한 클록의 press를 만든다.

1 kHz 주 클록에서 20샘플은 20 ms다. 버튼을 누르고 놓는 한 동작이 enable 한 번이 된다.

ASYNC_REG는 동기화 레지스터를 도구에 알리는 속성이다. 버튼 디바운스는 별도의 안정 시간 검사 로직이 수행한다.

보드 연결과 스위치 순서

항목	역할
B6 / clk	1 kHz 주 클록
K4 / rst	네 비트 초기화
N8 / button	한 단계 시프트
DIP SW1 / sw[7]	serial_in
LED[3:0]	value[3:0]
LED[7:4]	항상 0000

DIP SW1-SW8은 sw[7]-sw[0] 순서다. SW1을 정한 뒤 입력이 안정되면 N8을 누르고 놓는다.

테스트벤치의 여덟 검사

검사 시각	동작	value (16진수)
6 ns	reset	0
16 ns	1 입력	8
26 ns	0 입력	4
36 ns	enable=0, 입력은 1	4 유지
46 ns	enable=1, 1 입력	A
56 ns	0 입력	5
96 ns	0을 네 번 더 입력	0
106 ns	rst=1, 입력은 1	0

클록 에지와 검사 시각

TB는 always #5로 10 ns 주기의 클록을 만든다. 실제 보드의 1 kHz와 달라도 코어의 에지별 동작을 빠르게 검사할 수 있다.

step은 @(posedge clk) 뒤 #1을 기다린다. 5 ns 에지에서 초기화하고 6 ns에 갱신된 값을 비교한다.

값이 맞지 않으면 LAB2_FAIL과 \$fatal로 중단한다. 여덟 검사를 모두 통과해야 LAB2_PASS를 출력한다.

tb_shift_register4.sv · 1-16행

sim/tb_shift_register4.sv를 열고 해당 구간을 직접 입력한다.

```
sim > tb_shift_register4.sv > ...
1 | timescale 1ns/1ps
2 | module tb_shift_register4;
3 | reg clk=0, rst=1;
4 | integer checks=0;
5 | always #5 clk=~clk;
6 | task step; begin @(posedge clk); #1; end endtask
7 | task check(input condition, input [8*100-1:0] description);
8 | begin
9 |     checks=checks+1;
10 |    if (condition != 1'b1) begin
11 |        $display("LAB2_FAIL %0s time=%0t",description,$time);
12 |        $fatal(1,"check failed");
13 |    end
14 | end
15 | endtask
16 | task finish;
```

tb_shift_register4.sv · 17-32행

sim/tb_shift_register4.sv를 열고 해당 구간을 직접 입력한다.

```
17 begin
18     $display("LAB2_PASS shift_register4 checks=%0d",checks);
19     $finish;
20 end
21 endtask
22 initial begin $dumpfile("wave.vcd"); $dumpvars(0,tb_shift_register4); end
23 initial begin #100000; $fatal(1,"watchdog timeout"); end
24 reg enable=0, serial_in=0;
25 wire [3:0] value;
26 shift_register4 dut(clk,rst,enable,serial_in,value);
27 initial begin
28     step; check(value===0,"reset"); rst=0; enable=1;
29     serial_in=1; step; check(value===4'b1000,"input enters MSB");
30     serial_in=0; step; check(value===4'b0100,"move toward LSB");
31     enable=0; serial_in=1; step; check(value===4'b0100,"hold");
32     enable=1; step; check(value===4'b1010,"retain old stage values");
```

tb_shift_register4.sv · 33–38행

sim/tb_shift_register4.sv를 열고 해당 구간을 직접 입력한다.

```
33     serial_in=0; step; check(value===4'b0101,"four bit history");
34     repeat(4) step; check(value===0,"flush with four zeros");
35     serial_in=1; rst=1; step; check(value===0,"reset priority");
36     finish;
37 end
38 endmodule
```

XDC 전체를 직접 작성

constraints/lab2_shift_register.xdc를 만들고 다음 세 화면을 모두 입력한다.

PACKAGE_PIN의 포트 이름·비트 번호를 lab2_shift_register.v와 대조한다. B6의 주기는 1,000,000 ns다.

Icarus 코어 TB는 XDC를 사용하지 않는다. PASS만으로 실제 핀 연결이나 버튼 동작이 검증되지는 않는다.

lab2_shift_register.xdc · 1-16행

constraints/lab2_shift_register.xdc를 열고 해당 구간을 직접 입력한다.

```
1  # Combo II-DLD S75 / xc7s75fgga484-1 / main clock 1 kHz.
2  # Pin source and array polarity: evidence/board-pin-provenance.json.
3  set_property PACKAGE_PIN B6 [get_ports {clk}]
4  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
5  set_property PACKAGE_PIN K4 [get_ports {rst}]
6  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
7  set_property PACKAGE_PIN N8 [get_ports {button}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {button}]
9  set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
11 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
13 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
15 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
```

lab2_shift_register.xdc · 17-32행

constraints/lab2_shift_register.xdc를 열고 해당 구간을 직접 입력한다.

```
17  set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
18  set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
19  set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
20  set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
21  set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
22  set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
23  set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
24  set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
25  set_property PACKAGE_PIN N5 [get_ports {led[0]}]
26  set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
27  set_property PACKAGE_PIN M1 [get_ports {led[1]}]
28  set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
29  set_property PACKAGE_PIN M3 [get_ports {led[2]}]
30  set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
31  set_property PACKAGE_PIN M7 [get_ports {led[3]}]
32  set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
```

lab2_shift_register.xdc · 33-42행

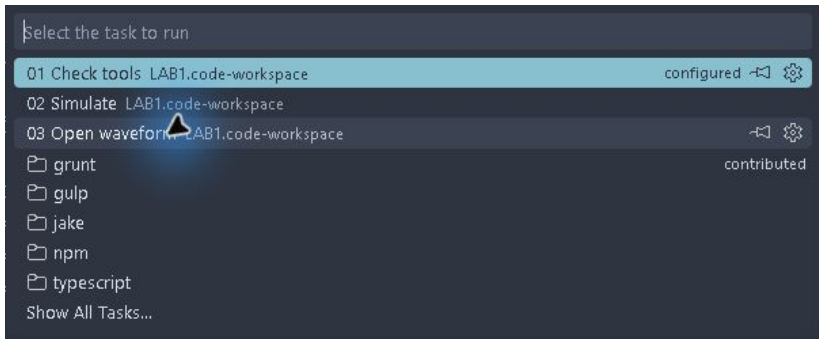
constraints/lab2_shift_register.xdc를 열고 해당 구간을 직접 입력한다.

```
33 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
34 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
35 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
36 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
37 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
39 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
40 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
41 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
42 set_false_path -from [get_ports {rst button sw[*]}]
```

VS Code에서 실행

1. File → Save All로 모든 파일을 저장한다.
2. Terminal → Run Task... → 01 Check tools.
3. 같은 메뉴에서 02 Simulate. Ctrl+Shift+B도 기본 시뮬레이션 작업을 실행한다.
4. 오류가 있으면 첫 파일·행 번호부터 확인한다. 마지막 줄만 보고 이전 PASS를 재사용하지 않는다.

작업 메뉴의 세 번호



공통 작업 메뉴: 01 도구 확인 → 02 시뮬레이션 → 03 파형 열기 순서다.

시프트 레지스터 검사 통과

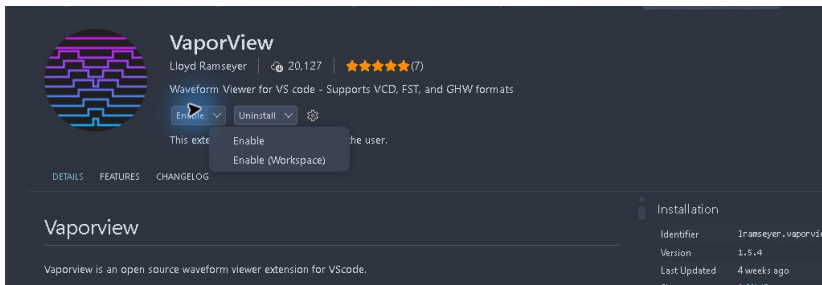
```
Executing task in folder PROJECT: C:/Users/UOS/AppData/Local/Programs/Python/Python312/python.exe tools/lab1.py simulate tb_shift_register4.sv

RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\UOS\Documents\ece2_26_2_2\tmp\lab2-students-0913\lab2_04_shift_register\build\sim\run -a25c0d50be124c3993e766bf61dba66c\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS shift_register4 checks=8
C:\Users\UOS\Documents\ece2_26_2_2\tmp\lab2-students-0913\lab2_04_shift_register\sim\tb_shift_register4.sv:19: $finish called at 106000 (1ps)

시뮬레이션 실행 완료. 진리표 자기검사 파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\UOS\Documents\ece2_26_2_2\tmp\lab2-students-0913\lab2_04_shift_register\build\sim\wave.vcd
Terminal will be reused by tasks, press any key to close it.
```

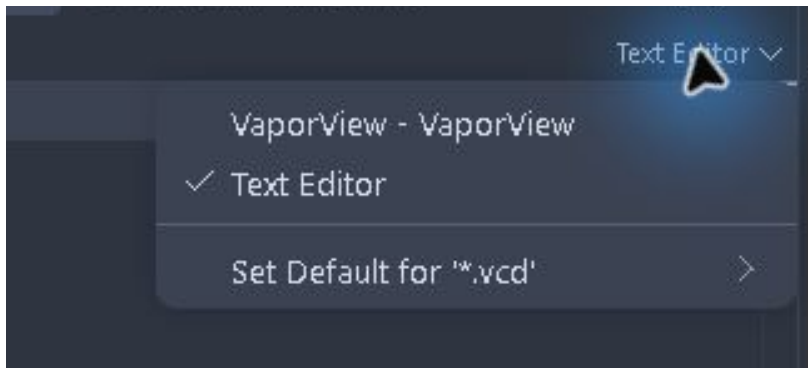
LAB2_PASS shift_register4 checks=8과 106000 ps(106 ns) 종료를 확인한다.

VaporView 활성화



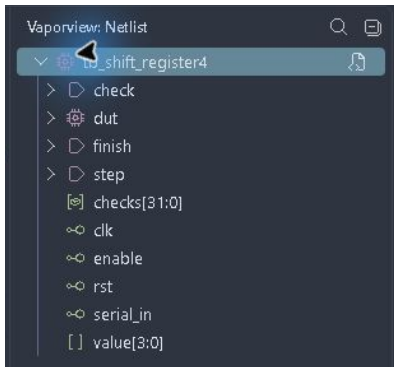
Extensions → @installed vaporview. 꺼져 있다면 Enable 옆 화살표 → Enable (Workspace)를 선택한다.

VCD를 파형 편집기로 열기



build/sim/wave.vcd를 연다. 텍스트로 보이면 오른쪽 위 편집기 선택 메뉴에서 VaporView를 선택한다.

다섯 신호 추가



Netlist View에서 tb_shift_register4를 펼친다. clk·rst·enable·serial_in·value를 차례로 더블클릭한다.

0-106 ns 전체 파형



Zoom Fit으로 전체 구간을 맞추고 Time Units → ns. 신호 이름이 잘리면 이름 열 경계를 오른쪽으로 넓힌다.

파형 읽기: 입력과 유지

15 ns 에지에서 입력 1이 bit 3에 들어가 value=8이 된다. 25 ns 에지에는 입력 0과 이전 100의 연결로 value=4다.

35 ns에는 serial_in=1이어도 enable=0이므로 value=4를 유지한다. 이 구간으로 enable의 역할을 확인한다.

45 ns에는 enable=1이므로 $\{1,010\}=1010$, 즉 A가 된다. 55 ns에는 $\{0,101\}=0101$, 즉 5가 된다.

파형 읽기: 비우기와 초기화

65, 75, 85, 95 ns 에지에 0을 넣으면 $5 \rightarrow 2 \rightarrow 1 \rightarrow 0 \rightarrow 0$ 순서다. TB는 네 에지를 모두 지난 96 ns에 마지막 0을 검사한다.

105 ns에는 serial_in과 enable이 모두 1이지만 rst가 우선하여 value=0이다. 비교는 106 ns에 끝난다.

처음 0-5 ns의 X는 아직 리셋 상승 에지를 거치지 않은 reg의 미정 상태다. 첫 리셋 에지 이후까지 X가 남으면 원인을 찾아야 한다.

수정 실험: 시프트 방향 뒤집기

1. 정상 로그와 wave.vcd를 보관한다.
2. shift_register4.v의 {serial_in,value[3:1]}을 {value[2:0],serial_in}으로 바꾼다. TB 기대값은 유지한다.
3. 저장 → 02 Simulate. 16 ns의 input enters MSB 검사가 실패하는지 확인한다.
4. 첫 입력 1에서 기대값 1000과 변경된 회로의 0001을 비교한다.
5. RTL을 원복·저장·재실행하고 checks=8, 106 ns 종료를 확인한다.

실험 전 레포트

1. enable·rst별 다음 상태 표와 $8 \rightarrow 4 \rightarrow A \rightarrow 5$ 계산을 작성한다.
2. 직접 입력한 RTL·TB·simulation.json·XDC의 역할을 설명한다.
3. 정상·변경·원복의 로그, 검사 시각, 기대값·실제값을 비교한다.
4. VS Code 전체 파형과 enable 유지·리셋 우선 구간을 넣어 해석한다.
5. 보드의 SW1·N8·K4 조작과 예상 LED를 정리한 뒤 Vivado를 시작한다.

Vivado에서 같은 원본 연결

VS Code 정상·변경·원복 검사와 실험 전 레포트를 마친 뒤 시작한다.

설계 Top은 lab2_shift_register, 시뮬레이션 Top은 tb_shift_register4다.

프로젝트 이름과 위치

Project name:

Project location:

☐ Create project subdirectory

Project will be created at: C:/Users/UOS/Documents/ece2_26_2/tmp/lab2-students-0913/lab2_04_shift_register/vivado

File → Project → New. 이름은 lab2_shift_register, 위치는 자신의 lab2_04_shift_register/vivado. Create project subdirectory 해제 → Next.

RTL 프로젝트 선택

- ☒ RTL Project
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL
- ☒ Do not specify sources at this time
- ☐ Project is an extensible Vitis platform

RTL Project와 Do not specify sources at this time을 선택한다. Next.

부품 검색과 선택

Parts	Boards				
Q· xc7s75fgga484-1		 	(3 matches)		
Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops	Block RAMs
xc7s75fgga484-1	484	338	48000	96000	90
xc7s75fgga484-1IL	484	338	48000	96000	90
xc7s75fgga484-1Q	484	338	48000	96000	90

Parts 검색란에 xc7s75fgga484-1을 입력한다. 같은 이름의 행을 선택한다. -1IL, -1Q와 구분한다. Next.

생성 요약 확인

New Project Summary

- A new RTL project named 'lab2_shift_register' will be created.
- The default part and product family for the new project:
Default Part: xc7s75fgga484-1
Family: Spartan-7
Package: fgga484
Speed Grade: -1

이름·Spartan-7·fgga484·-1을 확인하고 Finish. 이전 프로젝트가 열려 있으면 저장 상태 확인 후 닫는다.

빈 프로젝트 확인

Settings	Edit
Project name	lab2_shift_register
Project location	C:/Users/UOS/Documents/ece2_26_2_2/tmp/lab2-students-0913/lab2_04_shift_register/
Product family	Spartan-7
Project part	xc7s75fgga484-1
Top module name	Not defined
Target language	Verilog
Simulator language	Mixed
Target Simulator	Vivado Simulator

Project Summary에서 부품을 확인한다. 아직 소스를 넣지 않았으므로 Top module은 Not defined다.

설계 소스 추가

- ☐ Add or create constraints
- ☒ Add or create design sources
- ☐ Add or create simulation sources

왼쪽 Add Sources → Add or create design sources → Next → Add Files.

RTL 세 파일 선택



자신의 src 폴더를 연다. input_frontend.v·lab2_shift_register.v·shift_register4.v를 모두 선택하고 OK.

원본 RTL 참조

	Index	Name	Library	Version	Location
●	1	input_frontend.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp/
●	2	lab2_shift_register.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp/
●	3	shift_register4.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp/

- ☒ Scan and add RTL include files into project
- ☐ Copy sources into project

세 파일을 확인한다. Copy sources into project를 해제하고 Finish. VS Code에서 수정하는 원본과 연결한다.

테스트벤치 추가

- ☐ Add or create constraints
- ☐ Add or create design sources
- ☒ Add or create simulation sources

Add Sources → Add or create simulation sources → Next → Add Files.
sim/tb_shift_register4.sv를 선택한다.

TB 등록 확인

Specify simulation set: sim_1

+	-	↑	↓		
	Index	Name	Library	Version	Location
●	1	tb_shift_register4.sv	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp/

- ☒ Scan and add RTL include files into project
- ☐ Copy sources into project
- ☐ Add sources from subdirectories
- ☒ Include all design sources for simulation

sim_1과 Include all design sources for simulation을 유지한다. Copy sources into project를 해제하고 Finish.

XDC 추가

- ☒ Add or create constraints
- ☐ Add or create design sources
- ☐ Add or create simulation sources

Add Sources → Add or create constraints → Next → Add Files.
constraints/lab2_shift_register.xdc를 선택한다.

XDC 원본 참조

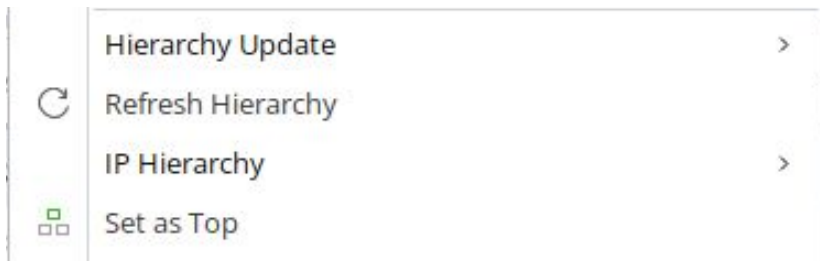
Specify constraint set: constrs_1 (active)

+ - ↑ ↓	
Constraint File	Location
lab2_shift_register.xdc	C:\Users\UOS\Documents\ece2_26_2\tmp\lab2-students-0913\lab2_04_shift_register\constraints

☐ Copy constraints files into project

Copy constraints files into project를 해제하고 Finish. 핀 변경도 같은 원본 파일에 반영한다.

테스트벤치를 Top으로



Sources → Simulation Sources → sim_1을 펼친다. tb_shift_register4
우클릭 → Set as Top.

서로 다른 Top 확인

Simulation Sources (2)

sim_1 (2) (active)

> tb_shift_register4 (tb_shift_register4.sv) (1)

> lab2_shift_register (lab2_shift_register.v) (2)

Hierarchy

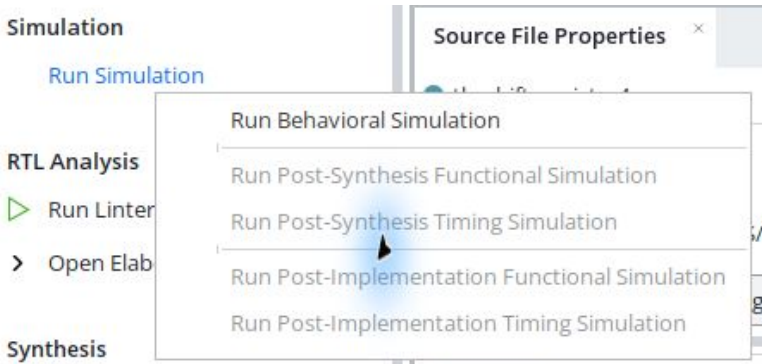
Libraries

Compile Order

Project name	lab2_shift_register
Project location	C:/Users/UOS/Documents
Product family	Spartan-7
Project part	xc7s75fgga484-1
Top module name	lab2_shift_register

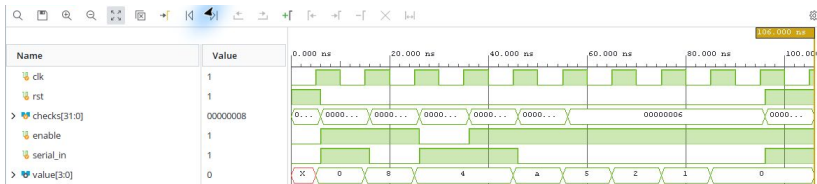
tb_shift_register4가 굵게 표시된다. Project Summary의 설계 Top은 lab2_shift_register로 유지한다.

XSim 실행



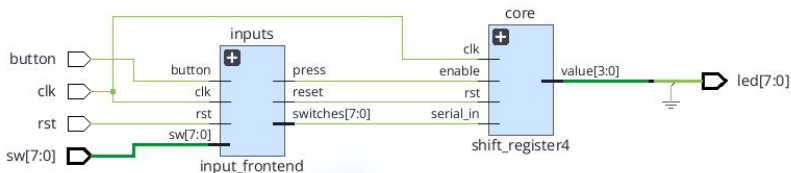
왼쪽 Run Simulation → Run Behavioral Simulation. 컴파일과 elaboration이 끝날 때까지 기다린다.

XSim 전체 파형



파형 탭을 더블클릭해 확대하고 Zoom Fit을 누른다. 106 ns 종료와 checks=8을 확인한다. value는 0 → 8 → 4 유지 → A → 5 → 2 → 1 → 0이다.

보드 회로 연결 확인



Open Elaborated Design → OK. 입력 처리 회로의 **press**가 **enable**, **switches[7]**이 **serial_in**으로 연결되는지 확인한다.

LED 핀 확인

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std
v All ports (19)						
v led (8)	OUT			✓	35	LVC MOS33* v
✓ led[7]	OUT		L4 v	✓	35	LVC MOS33* v
✓ led[6]	OUT		M4 v	✓	35	LVC MOS33* v
✓ led[5]	OUT		M2 v	✓	35	LVC MOS33* v
✓ led[4]	OUT		N7 v	✓	35	LVC MOS33* v
✓ led[3]	OUT		M7 v	✓	35	LVC MOS33* v
✓ led[2]	OUT		M3 v	✓	35	LVC MOS33* v
✓ led[1]	OUT		M1 v	✓	35	LVC MOS33* v
✓ led[0]	OUT		N5 v	✓	35	LVC MOS33* v

I/O Planning → I/O Ports 탭 더블클릭 → Expand All. led[7:0]의 Package Pin과 LVC MOS33을 XDC와 대조한다.

스위치 핀 확인

▼  sw (8)	IN			✓	34	LVC MOS33*	▼	
 sw[7]	IN		Y1	▼	✓	34	LVC MOS33*	▼
 sw[6]	IN		W3	▼	✓	34	LVC MOS33*	▼
 sw[5]	IN		U2	▼	✓	34	LVC MOS33*	▼
 sw[4]	IN		T1	▼	✓	34	LVC MOS33*	▼
 sw[3]	IN		W4	▼	✓	34	LVC MOS33*	▼
 sw[2]	IN		W1	▼	✓	34	LVC MOS33*	▼
 sw[1]	IN		V4	▼	✓	34	LVC MOS33*	▼
 sw[0]	IN		U4	▼	✓	34	LVC MOS33*	▼

같은 표의 sw[7:0]을 확인한다. 이 회로의 직렬 입력은 sw[7]이며 보드의 SW1에 대응한다.

버튼·클록·리셋 핀 확인

▼ ⓘ Scalar ports (3)							
button	IN		N8	▼	✓	35	LVC MOS33* ▼
clk	IN		B6	▼	✓	36	LVC MOS33* ▼
rst	IN		K4	▼	✓	35	LVC MOS33* ▼

Scalar ports의 button=N8, clk=B6, rst=K4와 LVC MOS33을 확인한다. LED 8개·스위치 8개·단일 포트 3개, 총 19개다.

합성 실행

Launch directory

 <Default Launch Directory>



Options



Launch runs on local host:

Number of jobs:

4



Generate scripts only

왼쪽 Run Synthesis. Launch runs on local host와 Number of jobs를 확인하고 OK. 이 실행은 jobs=4다.

합성 완료

 Synthesis successfully completed.

Next

- ☒ Run Implementation
- ☐ Open Synthesized Design
- ☐ View Reports

Synthesis successfully completed를 확인한다. Run Implementation 선택
→ OK.

구현 실행

Launch directory

 <Default Launch Directory>



Options



Launch runs on local host:

Number of jobs:

4



Generate scripts only

Launch runs on local host → OK. 구현 중에는 RTL과 XDC를 수정하지 않는다.

구현 완료

 Implementation successfully completed.

Next

- ☐ Open Implemented Design
- ☒ Generate Bitstream
- ☐ View Reports

Implementation successfully completed를 확인한다. Generate Bitstream 선택 → OK.

비트스트림 생성 실행

Launch directory

 <Default Launch Directory>



Options



Launch runs on local host:

Number of jobs:

4



Generate scripts only

Launch runs on local host와 jobs를 확인하고 OK. write_bitstream 완료까지 기다린다.

비트스트림 생성 완료

 Bitstream Generation successfully completed.

Next

- ☒ Open Implemented Design
- ☐ View Reports
- ☐ Open Hardware Manager
- ☐ Generate Memory Configuration File

Bitstream Generation successfully completed 확인. Open Implemented Design → OK. 이전 Elaborated Design을 닫을지 물으면 Yes.

타이밍 확인

Design Timing Summary

Setup		Hold	
Worst Negative Slack (WNS)	999997.000 ns	Worst Hold Slack (WHS)	0.122 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	27	Total Number of Endpoints	27

Timing → Design Timing Summary. Setup·Hold의 slack과 Number of Failing Endpoints를 확인한다.

검사 결과와 생성 파일

XSim: LAB2_PASS shift_register4 checks=8, 106 ns 종료. Icarus와 같은 TB와 원복한 RTL을 사용했다.

생성 파일:

vivado/lab2_shift_register.runs/impl_1/lab2_shift_register.bit.

GUI 합성·구현·bit 생성까지 확인했다. 다음 장치 기록과 보드 촬영은 학생이 실습 장비에서 수행한다.

타이밍과 경고 해석

WNS=999997.000 ns, WHS=0.122 ns. Setup·Hold 실패 endpoint는 각각 0이다. 주 클럭 제약은 1,000,000 ns다.

TIMING-18: LED의 외부 출력 지연을 지정하지 않았다. 내부 타이밍 통과를 모든 외부 입출력 타이밍 보장으로 해석하지 않는다.

CFGBVS-1: 설정 बैंक 전압 속성이 없다. 보드 회로도와 설정 전압을 확인하여 값을 정한다. 경고만 없애려고 임의의 전압을 입력하지 않는다.

장치에 기록

1. 보드 전원·JTAG·주 클록 1 kHz를 확인한다.
2. Open Hardware Manager → Open Target → Auto Connect.
3. 장치 우클릭 → Program Device → 방금 생성한 lab2_shift_register.bit → Program.
4. K4로 초기화한다. SW1을 먼저 정하고 N8을 누르고 놓는다.

보드에서 입력 이력 확인

조작	LED[7:0] (16진수)
K4 초기화	00
SW1=1, N8 한 번	08
SW1=0, N8 한 번	04
SW1=1, N8 한 번	0A
SW1=0, N8 한 번	05
SW1=0, N8 네 번	02 → 01 → 00 → 00

N8을 누르지 않고 SW1만 바꾸면 LED 값은 유지되어야 한다. 이 단계는 학생이 실제 장비에서 확인한다.

실험 후 레포트

1. 같은 TB의 XSim PASS·106 ns 종료·파형을 Icarus와 비교한다.
2. 부품·핀·클록·합성·구현·bit 파일과 타이밍·경고를 기록한다.
3. 실제 Program Device 화면과 보드 전체·LED 사진을 넣는다.
4. SW1 입력과 N8 조작으로 비트가 이동하고 K4로 초기화되는 영상을 촬영한다.
5. 소스·전후 레포트·사진·영상 링크를 GitHub README에서 연결한다.